6

BÖLÜM: THREAD'LER, İŞLETIM SISTEMI ÇEKIRDEĞI & JMM TEMELLERI | MODULE: THREADS, OS KERNELS & JAVA MEMORY MODEL (JMM) FUNDAMENTALS

Thread Oluşturma Paradigmaları: Thread, Runnable, Callable & Java 25 Builder

Thread Creation Paradigms: Thread, Runnable, Callable & Modern Java 25 Builders

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
JAVA THREAD OLUŞTURMA EVRİMİ (JDK 1.0 ⟶ JDK 25)

1. ESKİ USUL (Kalıtım / Kötü):
   class Worker extends Thread { public void run() { ... } }
   ❌ Java tekli kalıtımını harcar, Thread yaşam döngüsü ile iş mantığı birbirine karışır!

2. AYRIŞTIRILMIŞ İŞ PARÇACIĞI (Composition / Standart):
   Runnable task = () -> { ... }; // Geriye değer dönemez, Exception fırlatamaz!
   Callable<String> task = () -> "Sonuç"; // Değer döner, Checked Exception fırlatabilir!

3. MODERN JAVA 25 BUILDER PATTERN (JDK 21+ & 25):
   Thread.ofPlatform().name("worker-", 1).start(task);
   Thread.ofVirtual().name("loom-", 1).start(task);
   ✓ Tek bir arayüzle hem ağır OS thread'i hem de hafif Virtual Thread açabilme!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eskiden bir kamyon almak istediğinizde şoförü de kamyonun motoruna kaynaklanmış olarak alırdınız (extends Thread). Artık kamyon başka (Thread), şoförün yapacağı iş emri başkadır (Runnable/Callable)! Runnable, 'Git depoyu süpür' diyen ama sana cevap vermeyen bir emirdir. Callable ise 'Kasada kaç para var say ve bana söyle' diyen, sana bir sonuç (Future) getiren emirdir. Java 25'te ise artık 'Bana 1 numaralı platform işçisini ver' veya 'Bana uçan sanal işçi ver' diyebildiğimiz harika bir sipariş menümüz (Thread.ofPlatform() / Thread.ofVirtual()) vardır!

🇬🇧 ENGLISH (ELI5)

In Java 1.0, to buy a delivery truck, you had to weld the human driver directly to the chassis (`extends Thread`). Modern engineering separates the vehicle (Thread) from the cargo assignment (Runnable/Callable). `Runnable` is a 'Fire-and-forget' chore returning `void`. `Callable\<V>` is an order returning an explicit computed value and capable of throwing checked exceptions. In modern Java 25, the fluent `Thread.ofPlatform()` and `Thread.ofVirtual()` builders allow seamlessly stamping out heavyweight OS threads or lightweight user-mode fibers!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Java 25 Thread Builder Mimarisi: 1) `Thread` sınıfından türetmenin zararları: Object Oriented prensiplerine aykırıdır; 'İş' (Task) ile o işi 'Çalıştıran Motor' (Worker) birbirine sıkı sıkıya bağlanır. 2) `Runnable` vs `Callable`: Runnable `run()` metodu `void` döner ve checked exception fırlatamaz; Callable `call()` ise generik `V` döner ve `throws Exception` imzasına sahiptir. 3) Modern `Thread.Builder`: Java 21 ile gelen ve Java 25'te standartlaşan `Thread.ofPlatform()` ve `Thread.ofVirtual()`, thread konfigürasyonunu (isim öneki, daemon durumu, priority, UncaughtExceptionHandler) fluent API ile güvenle kurar.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Architectural Analysis of Thread Factories: 1) The Fallacy of Subclassing: Inheritance couples the task logic with OS thread scheduling state, violating the Single Responsibility Principle (SRP). 2) `Runnable` vs `Callable\<V>` Contract: `Runnable.run()` is a void-returning, non-throwing functional interface. `Callable\<V>.call()` enables return value propagation and checked exception handling, foundational to Executor frameworks. 3) Java 25 Fluent Builders: `Thread.ofPlatform()` and `Thread.ofVirtual()` provide type-safe builders for thread naming, daemon flags, uncaught exception handlers, and thread factory generation (`ThreadFactory`).

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Modern Thread Builders ve Callable Entegrasyonu
import java.util.concurrent.Callable;
import java.util.concurrent.FutureTask;

void main() throws Exception {
    // 1. Modern Java 25 Platform Thread Builder
    Thread platformThread = Thread.ofPlatform()
        .name("payment-worker-", 1)
        .daemon(false)
        .start(() -> IO.println("Platform thread çalıştı: " + Thread.currentThread()));
    platformThread.join();

    // 2. Callable ile değer döndüren iş parçacığı
    Callable<Integer> calculateTaxTask = () -> {
        IO.println("Vergi hesaplanıyor...");
        return 1850;
    };

    var futureTask = new FutureTask<>(calculateTaxTask);
    Thread.ofPlatform().name("tax-calc").start(futureTask);

    // get() çağrısı hesaplama bitene kadar bekler (Blocking)
    int result = futureTask.get();
    IO.println("Hesaplanan Vergi: " + result + " TL");
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: 'Thread.run() çağırmak ile Thread.start() çağırmak arasındaki fark nedir?' Yanıt: 'start() metodu JVM üzerinden işletim sistemine yeni bir thread tahsis ettirir (OS thread) ve o yeni thread üzerinde run() metodunu tetikler. Doğrudan run() çağrılırsa yeni thread açılmaz, kod o anki mevcut thread içinde senkron çalışır!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Classic Phone Screen Trap: 'What happens if you invoke `thread.run()` instead of `thread.start()`?' Answer: '`start()` prompts the JVM to allocate an underlying OS kernel thread and invoke `run()` asynchronously. Calling `run()` directly merely executes the method synchronously on the current caller's thread, bypassing multithreading entirely.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Thread sınıfını extend etmek yerine Runnable/Callable kompozisyonu kullanılmalıdır.
- Callable geriye değer dönebilir ve Exception fırlatabilir; Runnable dönemez.
- Java 25'te Thread.ofPlatform() ve Thread.ofVirtual() standart builder'dır.

🎯 Key Takeaways for the Interview

- Favor composition (Runnable/Callable) over inheriting from Thread.
- Callable\<V> yields return values and throws checked exceptions; Runnable returns void.
- Java 25 fluent builders Thread.ofPlatform() / ofVirtual() unify thread instantiations.